High-Level Design (HLD)

Project- RAG-Based Customer Support Assistant

1. System Overview

Problem Definition

Modern customer support teams are overwhelmed by high volumes of repetitive inquiries regarding policies, product manuals, and internal documentation. While traditional chatbots attempt to solve this, they often lack specific context, resulting in inaccurate or generic answers (hallucinations). Furthermore, when a user's problem is complex or they become frustrated, automated systems often fail to gracefully escalate the issue to a human agent, leading to poor customer satisfaction.

There is a critical need for an intelligent, context-aware system that can ingest proprietary knowledge bases, accurately retrieve relevant information to answer user queries, and dynamically route complex or sensitive issues to a human operator.

Scope of the System

This project entails the design and implementation of a Retrieval-Augmented Generation (RAG) Customer Support Assistant. The system is designed to act as the first line of defense for customer inquiries by grounding a Large Language Model (LLM) in proprietary PDF documentation.

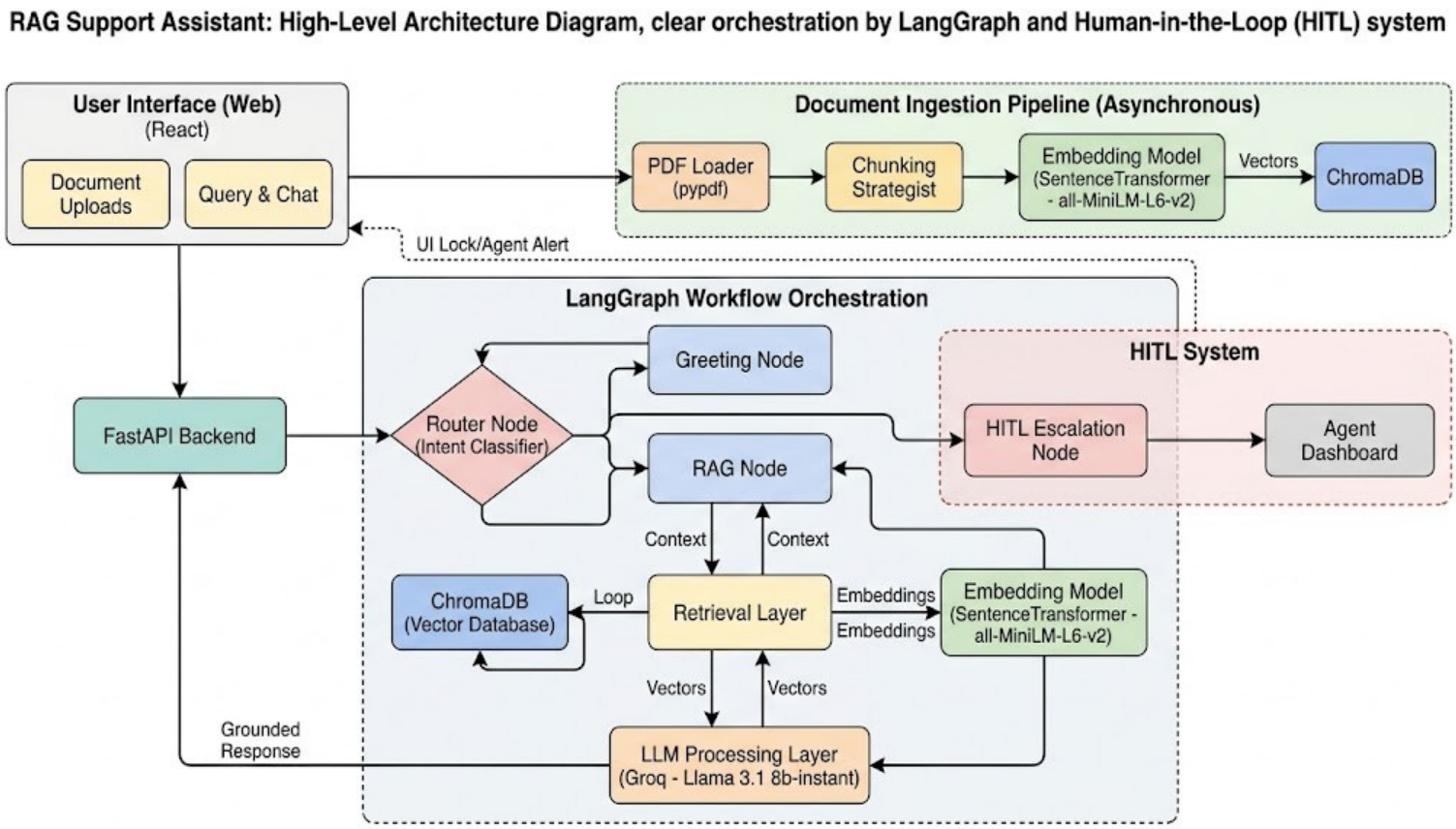
In-Scope:

- **Knowledge Ingestion:** The ability to upload, parse, and chunk PDF documents into meaningful text segments.
- **Vectorization & Storage:** Converting text chunks into semantic embeddings and storing them in a local vector database.
- **Semantic Retrieval:** Searching the vector database for chunks that are semantically relevant to a user's query.
- **Intent-Based Routing:** Analyzing incoming user messages to determine if the query is a standard question, a simple greeting, or a request for human escalation.
- **Contextual Generation:** Utilizing a fast LLM to generate accurate, polite customer support responses based strictly on the retrieved context.
- **Human-in-the-Loop (HITL) Simulation:** A workflow mechanism that pauses automated generation and alerts a human agent when the system detects frustration or explicit escalation requests.

- **Interactive UI:** A web-based interface (React/FastAPI) allowing users to upload documents, scope queries to specific files, and chat with the assistant.

2. Architecture Diagram

The system architecture is built on a modular RAG (Retrieval-Augmented Generation) framework, coordinated by a state-driven graph. It separates the heavy-duty document processing (Ingestion) from the real-time user interaction (Inference).



Components of the Architecture

A. User Interface (Web)

- **Role:** The primary entry point for both customers and administrators.
- **Functionality:** Built with React, it provides a dual-purpose interface: a document management dashboard for uploading PDFs and a conversational chat interface. It dynamically handles "Agent Lock" states when a human escalation is triggered.

B. Document Ingestion Pipeline

- **Role:** Pre-processes raw data into a machine-readable format.

- **Workflow:**

  1. **Loader:** Uses pypdf to extract text from byte streams.

  2. **Cleaner:** Removes boilerplate and excessive whitespace.

  3. **Chunker:** Implements recursive character splitting to create manageable 500-word segments with overlap.

C. Embedding System

- **Role:** Translates text into mathematical vectors.
- **Model:** Uses the all-MiniLM-L6-v2 Sentence Transformer.
- **Benefit:** This model provides a high-dimensional semantic representation (384 dimensions), allowing the system to understand that "How do I pay?" and "Billing methods" are semantically identical.

D. Vector Database (Chroma DB)

- **Role:** The system's long-term semantic memory.
- **Mechanism:** Stores document chunks as high-dimensional vectors. It supports Metadata Filtering, which is critical for our "Scope to File" feature, allowing the retrieval layer to ignore chunks from irrelevant documents.

E. Retrieval Layer

- **Role:** Context fetching.
- **Mechanism:** When a user asks a question, this layer vectorizes the query and performs a Cosine Similarity Search against Chroma DB. It returns the top 4 most relevant "source truths" to ground the LLM's response.

F. LLM Processing Layer

- **Role:** Reasoning and synthesis.
- **Model:** Groq-hosted Llama-3.1-8b-instant.
- **Dual Usage:** 1. **Classifier:** Acts as the "Brain" of the router to detect intent.

  2. **Generator:** Converts dry retrieved text into a polite, professional support response.

G. Workflow Orchestration (Langgraph)

- **Role:** The "Air Traffic Controller" of the logic.
- **Mechanism:** Instead of a linear script, LangGraph treats the conversation as a State Machine. It maintains the current Agent State and decides whether to move the query to the greeting_node, rag_node, or escalate_node based on the detected intent.

H. HITL System (Human-in-the-Loop)

- **Role:** Safety and complex resolution.
- **Mechanism:** A specialized node in the graph that triggers when user frustration is detected or a human is requested. It outputs a needs human: true flag which travels through the API to the UI, effectively pausing the AI and switching the channel to "Human Wait" mode.

3. Component Description

A. Document Loader

- **Tool:** pypdf (via custom Python logic).
- **Purpose:** Reads raw byte streams from uploaded PDFs, ignores unstructured/unreadable elements (like complex images), and extracts raw text page by page.

B. Chunking Strategy

- **Method:** Fixed-size text chunking with overlap (e.g., 500 words per chunk with a 50-word overlap).
- **Purpose:** Ensures that text fed to the vector database is small enough to maintain high semantic density, while the overlap prevents contextual boundaries (like sentences cut in half) from losing their meaning.

C. Embedding Model

- **Tool:** Hugging Face Sentence Transformer (Model: all-MiniLM-L6-v2).
- **Purpose:** Runs locally to convert text chunks and user queries into high-dimensional vector embeddings. This specific model is chosen because it is incredibly fast, runs without a GPU, and is highly optimized for semantic similarity tasks.

D. Vector Store

- **Tool:** ChromaDB (Persistent Client).
- **Purpose:** Stores the vectorized chunks alongside their metadata (specifically the filename). The filename metadata allows the system to perform targeted/scoped queries where the user asks a question about one specific document rather than the whole database.

E. Retriever

- **Method:** Cosine Similarity Search via ChromaDB query().
- **Purpose:** Takes the vectorized user query and retrieves the Top-K (e.g., Top 4) most semantically similar text chunks to use as context for the LLM.

F. Large Language Model (LLM)

- **Tool:** Groq API (llama-3.1-8b-instant).
- **Purpose:** Used for two tasks:

1. **Intent Classification:** A rapid zero-shot classification to determine where to route the user.
2. **Generation:** Synthesizing the retrieved context into a natural, conversational response that fits the Customer Support persona.

G. Graph Workflow Engine

- **Tool:** LangGraph (StateGraph).
- **Purpose:** Orchestrates the control flow. Instead of a linear script, the system uses a state machine (AgentState) containing the question, intent, answer, sources, and needs_human flag. It ensures the query is routed to the correct processing node before returning to the user.

H. Routing Layer (Intent Classifier)

- **Tool:** LLM-based JSON output.
- **Purpose:** The entry point of the LangGraph. It analyzes the user's message and forces the LLM to output a strict JSON classification: greeting, rag_query, or escalate.

I. Human-In-The-Loop (HITL) Module

- **Method:** State Flagging (needs_human = True).
- **Purpose:** When the Router triggers the Escalation Node, this module bypasses document retrieval. Instead, it updates the state graph with a flag that signals the React Frontend to lock the user interface, simulating a pause while a human agent connects to the session.

4. Data Flow

The data flow is divided into two primary lifecycles: the Ingestion Lifecycle (where documents become knowledge) and the Query Lifecycle (where questions become answers).

A. Ingestion Lifecycle (PDF → Vector)

1. **Transport:** User uploads a PDF via the React frontend; the file is sent as a byte stream to the FastAPI /upload endpoint.
2. **Extraction:** The system uses pypdf to parse the byte stream and extract raw text.
3. **Decomposition (Chunking):** Raw text is split into segments of 500 words with a 50-word overlap to ensure semantic continuity.
4. **Vectorization:** Each text chunk is passed through the all-MiniLM-L6-v2 model, resulting in a 384-dimensional vector.
5. **Persistence:** Vectors are stored in ChromaDB alongside metadata (filename and unique IDs) for future filtered retrieval.

B. Query Lifecycle (User Query → Response)

1. **Entry:** The user submits a question through the UI.
2. **State Initialization:** LangGraph initializes the AgentState with the user's question.
3. **Intent Classification (Routing):** The router_node sends the question to the LLM. If the intent is classified as escalate, the flow moves to the HITL node. If it is a greeting, it moves to the greeting node. Otherwise, it triggers the RAG node.
4. **Context Retrieval:** In the RAG node, the query is vectorized and used to perform a similarity search in ChromaDB. The top 4 most relevant chunks are returned.
5. **Grounded Generation:** The LLM receives the retrieved chunks and the user question. It synthesizes a response using ONLY the provided context.
6. **Exit:** The final answer is returned to the UI, and the frontend state is updated (including locking the UI if the needs_human flag is true).

5.Technology Choices

Technology	Choice	Why?
Vector Database	Chroma DB	Chosen for its "Local-first" nature. It is lightweight, requires no external cloud hosting for this project, and offers robust metadata filtering for scoping queries to specific files.
Workflow Engine	LangGraph	Unlike linear chains, LangGraph allows for Cycles and State Management. It provides a robust way to implement the "Router" logic and handle Human-in-the-Loop interruptions.
Inference Engine	Groq (Llama 3.1)	Groq's LPU architecture provides near-instant inference speeds. Llama 3.1 8B is used because it is highly capable of following strict "Answer only from context" instructions.
Embeddings	Sentence Transformers	The all-MiniLM-L6-v2 model is the industry standard for lightweight, local semantic search. It balances accuracy with extremely low latency.

6. Scalability Considerations

A. Handling Large Documents

- **Current Limit:** Very large PDFs may cause timeouts during the upload phase.
- **Scalability Fix:** Moving the ingestion process to a Background Task (using FastAPI's BackgroundTasks or Celery). This allows the user to continue using the app while the document is processed in the background.

B. Increasing Query Load

- **Current Status:** The system is synchronous per request.
- **Scalability Fix:** Deploying the FastAPI app using Uvicorn workers or behind a Load Balancer (like Nginx). Since ChromaDB is currently running in "Persistent Local" mode, moving to a client-server ChromaDB architecture would allow multiple API instances to share the same vector store.

C. Latency Concerns

- **Bottleneck:** Embedding the query on every request using the CPU.
- **Scalability Fix:** In a production environment, embedding models should be served via a dedicated Model Server (like NVIDIA Triton or TGI) or run on GPU instances to handle concurrent vectorization requests without slowing down the API.